4. UI/UX specification

Visual direction: sophisticated developer portfolio, not a generic template. Use a strong typographic hierarchy, asymmetric but controlled layout, thin borders, subtle shadows, a small number of accent colors, and real project descriptions. Do not fabricate screenshots or product logos.

Area	Desktop behavior	Mobile behavior
Header	Compact sticky header; wordmark/name left, anchor links and theme toggle right.	Small header; menu collapses cleanly; all controls reachable by keyboard and touch.
Hero	Two-column or editorial composition: name/role/summary and CTA on one side; abstract code/product visual on the other.	Single column; headline wraps naturally; CTA buttons stack or wrap.
About	Short summary with selected strengths; use readable line length.	One column; no tiny text or dense paragraphs.
Skills	Grouped chips/cards by language, core CS, backend, databases, app development, tools.	Wrap chips; avoid horizontal overflow.
Experience	Timeline or two structured role cards with company, dates and concise bullets.	Vertical timeline/cards; dates remain legible.
Projects	Three featured project cards with tech stack, summary, optional links only when supplied.	One card per row or compact swipe-free grid.
Education	Compact education block.	Compact single-column block.
Contact/footer	Clear email CTA, optional public profile links, back-to-top.	Large tap target for email; compact footer.

Design tokens

Token	Light mode	Dark mode
Page background	Warm off-white (#F8F9FC)	Deep ink (#0D111B)
Surface	White (#FFFFFF)	Raised charcoal (#151C2B)
Primary text	Ink (#172033)	Near-white (#F3F5FA)
Muted text	Slate (#596579)	Soft slate (#A8B2C4)
Accent	Blue (#315FEA) with teal sparingly	Bright blue (#7895FF) with teal sparingly
Borders	Cool light gray (#DCE2EC)	Subtle dark border (#2B3446)

Interaction rules

- Use smooth anchor scrolling unless reduced motion is enabled; sticky header must not cover section headings.
- Buttons and cards have clear hover, focus, active, and disabled states. Avoid constant motion or animation that distracts from reading.
- Use CSS-only decorative visual where possible; do not load large videos or heavy 3D assets in v1.
- Provide loading, empty, and error states for sheet-backed content. Never show a blank page on a fetch error.
- Use meaningful alt text for informative images and empty alt for decorative visuals.

5. Google Sheets content model

Create one Google spreadsheet with the tabs below. Publish only the data required by the public portfolio. Use stable IDs, explicit order values, and a consistent visibility field. Exact tab names should match the adapter configuration.

Tab	Required columns	Purpose
Profile	key, value, visible	Name, role, location, email, phone (optional), summary, availability text, CTA labels, profile links.
Sections	section_id, title, visible, order, description	Section-level show/hide and display ordering.
Skills	skill_id, category, name, visible, order	Skill group and individual skill chips.
Experience	experience_id, role, company, location, start_date, end_date, summary, bullets, visible, order	Roles, dates, and achievement/responsibility bullets.
Projects	project_id, name, description, tech_stack, link, image_url, visible, featured, order	Project cards; blank links/images must be handled safely.
Education	education_id, qualification, institution, year, details, visible, order	Education cards.
CustomSections	item_id, section_id, title, body, link_label, link_url, visible, order	Optional simple content blocks.

Visibility and data rules

- Use the exact value VISIBLE to display a row; treat blank, HIDDEN, or unknown status as hidden (fail closed).
- Use order as an integer; lower values appear first. Use stable IDs that do not change when copy is edited.
- Use ISO-like dates or human-readable dates consistently. For present roles, use end_date = Present.
- For bullets, either create one row per bullet or use a clearly documented delimiter. Prefer one row per bullet in a separate ExperienceBullets tab if parsing becomes error-prone.
- Do not use formulas that expose private source data. Do not include passwords, API keys, private phone numbers, personal addresses, or confidential company data.
- Only add external links after verifying that they belong to the owner and are intended for public display.

Resume content to seed

- Profile: Abhishek; Software Engineer | Flutter & Backend-Integrated Application Development; Gurugram, India; 2+ years of experience summary.
- Skills: Dart, Python, Java, C/C++; DSA, complexity analysis, OOP, DBMS; REST APIs, HTTP, JSON, Firebase Auth/Firestore/Cloud Messaging, Node.js, Django/Flask; MySQL, MongoDB, Firestore; Flutter, BLoC, Provider, React.js, Python Windows automation; Git, debugging, code review and AI-assisted development tools.
- Experience: Associate Software Engineer (Flutter), VCQRU, Gurugram, Mar 2025–Present; Flutter Intern (UI/UX), VCQRU, Aug 2024–Mar 2025.
- Projects: Loyalty Rewards App; Real-Time Chat App; Weather Forecast App.
- Education: MCA, Gurugram University, 2025.

Source: supplied resume. Do not invent metrics, awards, client names, project URLs, or claims not stated there.

6. SEO, security, accessibility and performance

SEO checklist

- Set a descriptive title and meta description, e.g. “Abhishek | Flutter & Software Engineer”.
- Use one clear H1 and logical H2 section headings; add canonical URL only when the final public URL is known.
- Include Open Graph title/description/image and a simple favicon.
- Ship robots.txt and sitemap.xml with the actual GitHub Pages URL; update if a custom domain is added.
- Keep the core identity, role, summary and important text in static HTML. Avoid making the entire page dependent on client-side sheet fetch.
- Use descriptive link text and meaningful project descriptions; avoid keyword stuffing.

Security and privacy

- A published Google Sheet is public. Treat every cell and published URL as public information.
- No secrets in frontend code, repository, spreadsheet, environment variables shipped to the browser, or static JSON.
- Use only public read-only content. If private editing is needed later, use Google Sheets UI directly with its own account permissions; never build a public admin route for v1.
- Escape/sanitize text before rendering; do not inject sheet values as raw HTML. Validate URLs and allow only http/https/mailto as appropriate.
- Avoid collecting visitor personal data. Use a mailto link for contact unless a secure form service is separately chosen.

Accessibility and performance

- Use semantic landmarks, keyboard-accessible navigation, focus-visible styles, descriptive labels, and WCAG-conscious contrast.
- Support prefers-reduced-motion; animation must never be required to understand content.
- Use optimized local assets, responsive image sizing, lazy-load below-the-fold images, and avoid layout shifts.
- Test on narrow mobile widths, keyboard only, light/dark themes, slow network, and sheet fetch failure.

7. Deployment and maintenance

GitHub Pages setup

- Create a GitHub repository for the portfolio and commit the source code.
- Configure Next.js for static export (output: 'export'); ensure the build emits an out/ directory.
- If deploying to a project site, set basePath and asset prefix appropriately. Test all asset URLs on the actual repository path.
- Add a GitHub Actions workflow to install dependencies, run lint/type checks, build, and publish the out/ artifact to GitHub Pages.
- Enable Pages in repository settings and verify the live URL, refresh behavior, mobile layout and browser console.
- For a custom domain, configure DNS and GitHub Pages domain settings only after the default URL works.

Google Sheets publishing workflow

- Create the spreadsheet and use the exact documented tab names and headers.
- Enter only public portfolio content and verify spelling, URLs and visibility values.
- Publish the required tabs to the web in CSV or another format selected by the adapter. Test the public URL in a private browser window.
- Set the published URLs in a public configuration file (not secrets). Document the republish steps when headers or tab names change.
- Update content by editing cells; the site should reflect published changes on refresh or within the configured cache interval.
- If Google changes endpoint behavior or CORS access fails, fall back to the bundled content and document the fix; do not silently show an empty section.

Acceptance checklist

- The site builds as a static export and deploys to GitHub Pages without server features.
- Light Mode is the default; Dark Mode toggle works and preference persists.
- All sections render correctly at 360 px mobile width and desktop widths without horizontal overflow.
- Changing a public Sheet value and refreshing updates the matching content; HIDDEN rows do not appear.
- Sheet fetch failure still displays useful fallback portfolio content.
- No secrets are present in repository, public sheet, or browser bundle.
- SEO metadata, favicon, robots.txt, sitemap.xml, and Open Graph metadata are present.
- Keyboard navigation, focus indicators, reduced motion, and readable contrast are verified.

8. Copy/paste implementation prompt for Google Antigravity

Give Antigravity this PDF and the completed Excel/Google Sheet template, then paste the following prompt. It should implement the project in small, testable stages and ask before making material scope changes.

```
You are the senior frontend engineer implementing Abhishek's personal portfolio.
Treat the attached System Design PDF and the provided spreadsheet as the source of truth.

GOAL
Build a premium, responsive, one-page developer portfolio with Light Mode as default,
a persistent Dark Mode toggle, tasteful motion, and content managed through a public,
read-only Google Sheet. Deploy as a static site on GitHub Pages with no paid services.

STACK
Next.js, TypeScript, Tailwind CSS, static export only. No server routes, middleware,
server actions, private API keys, database, authentication, or paid dependency.

REQUIREMENTS
1. Implement the architecture and UI rules in the PDF.
2. Keep core identity, role, summary and primary SEO content in static HTML.
3. Build a typed Google Sheets content adapter; validate rows, sort by order, and show
   only rows with visible == "VISIBLE". Add bundled fallback content and error handling.
4. Keep sheet parsing isolated in src/lib/content so the data source can be changed later.
5. Use semantic HTML, keyboard support, visible focus states, accessible contrast,
   reduced-motion support, and responsive layouts.
6. Never render sheet values as raw HTML. Validate external URLs.
7. Use only the resume facts supplied in the spreadsheet; do not invent achievements,
   project URLs, metrics, awards or employers.
8. Add title/meta description, Open Graph tags, favicon, robots.txt and sitemap.xml.
9. Add lint/type-check/build scripts and a GitHub Actions workflow for Pages deployment.
10. Include README instructions for local setup, sheet publishing, configuration,
    GitHub Pages deployment, troubleshooting and updating content.

WORK PROCESS
First inspect the provided files and summarize the planned implementation.
Then create a concise task checklist and implement in stages:
A) project scaffold and design tokens;
B) static layout and responsive sections;
C) theme toggle and accessibility;
D) sheet adapter, schema validation and fallback data;
E) SEO and assets;
F) tests/build/deployment workflow;
G) final audit against the acceptance checklist.
Run the available checks and fix errors. Report what was tested and what still needs
manual verification. Do not claim deployment succeeded unless it is actually verified.
Ask before adding paid services, analytics, a contact form backend, or extra scope.
```

Implementation note

The spreadsheet template is a separate deliverable. Before connecting it, confirm the published CSV URLs and validate the actual returned headers in a browser. Google Sheets publishing and CORS behavior must be tested; do not assume a URL will work solely because it opens in a browser.

End of specification